
PAC-MAN GAME

Architecture Report

Java · Console Game · OOP Design · April 2026

1. Introduction

Pac-Man is a classic Java console game where the player moves through a 9×9 walled grid collecting food pellets while avoiding a ghost. The project is built using five clean object-oriented classes, each with a single clear responsibility. The game ends when either all food is collected (win) or the ghost catches Pac-Man (lose).

2. Classes

PacmanGame	Entry point — initialises all objects and runs the main game loop.
GameBoard	Owns the 9×9 grid; handles wall detection, food logic, and board rendering.
Pacman	The player — handles WASD movement, food collection, and score tracking.
Ghost	The enemy — moves randomly each turn, rejected if it hits a wall or boundary.
Food	Simple data model storing the row and column of a food pellet on the grid.

Class Diagram

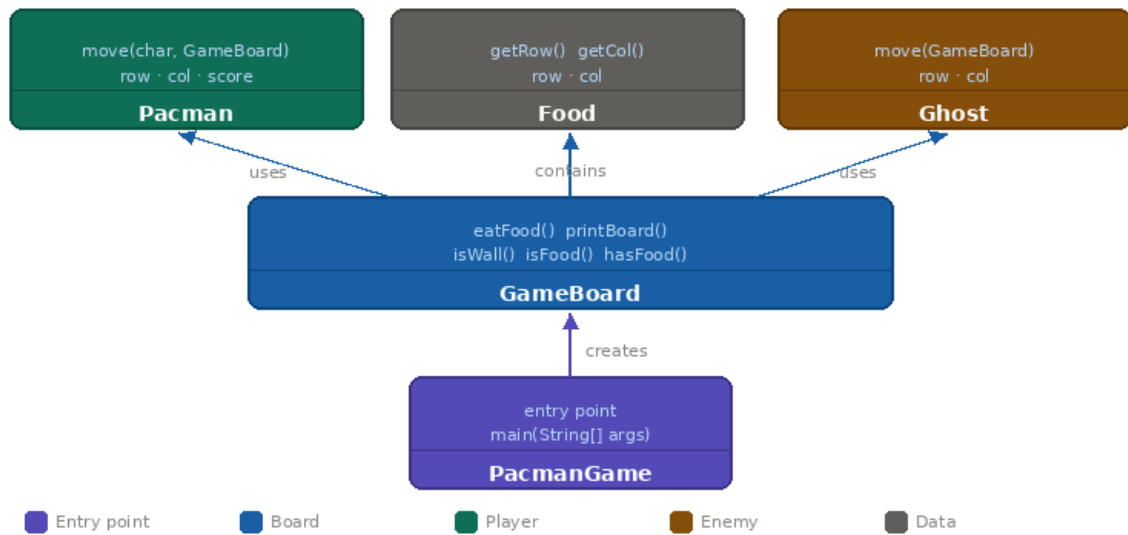


Figure 1 — Class relationships

3. How the Game Works

Each turn follows the same sequence: the board is rendered with Pac-Man (@) and Ghost (G) overlaid on the grid, then the player inputs a direction (W/A/S/D). Pac-Man moves if the target cell is within bounds and not a wall — collecting any food it lands on. The ghost then moves randomly. If they share the same cell, the player loses; if all food is collected, the player wins.

Game Loop Flowchart

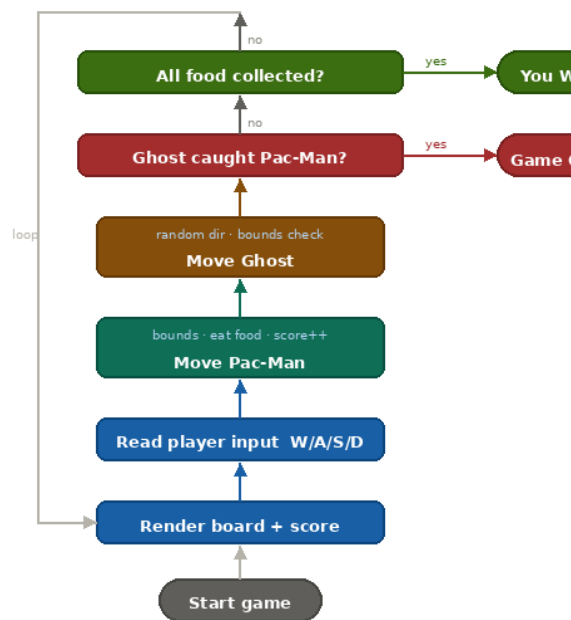


Figure 2 — Main game loop

Board Structure

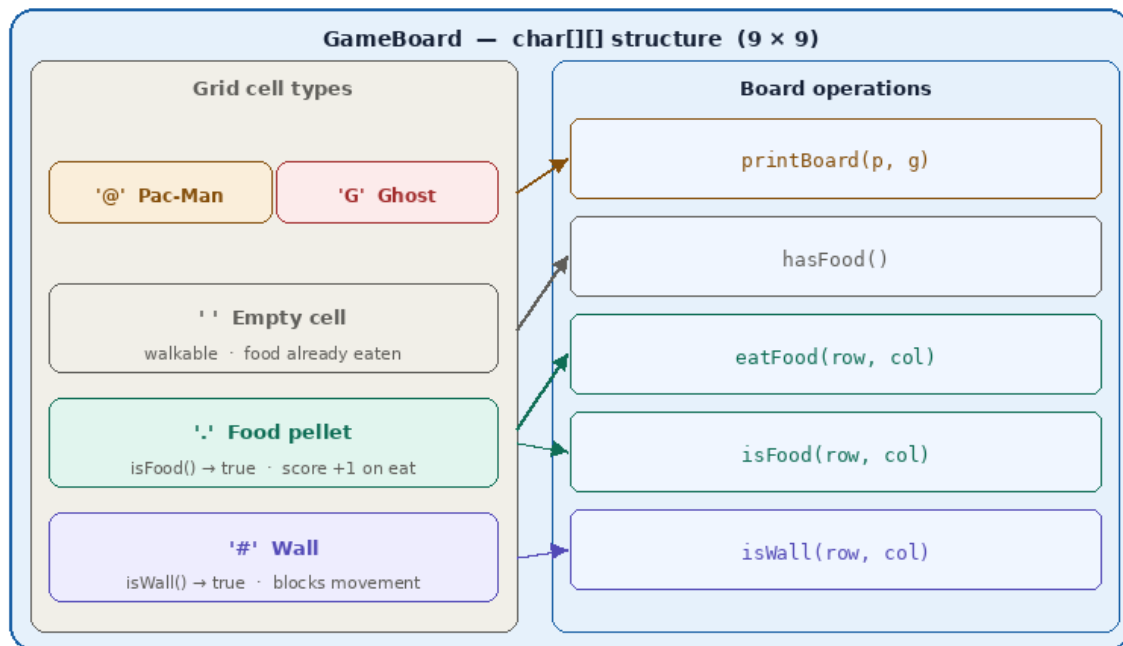


Figure 3 — GameBoard cell types and operations

4. Ideas for the Future

- Multiple ghosts — add more enemies that move independently each turn.
- Smarter AI — replace random movement with pathfinding (BFS or A*) so the ghost hunts the player.
- Multiple levels — introduce new board layouts that load progressively as the player wins.
- Power pellets — special cells that make Pac-Man temporarily invincible.
- High score board — save and display top scores between sessions.
- Graphical UI — replace the console output with a Java Swing or JavaFX window.

5. Conclusion

The Pac-Man game demonstrates solid object-oriented design in Java — five classes, each doing one job, working together cleanly. The code is easy to read, maintain, and extend. With the improvements applied (bug fixes, named constants, consistent style, and better end-game feedback), it serves as a strong foundation for future development.